

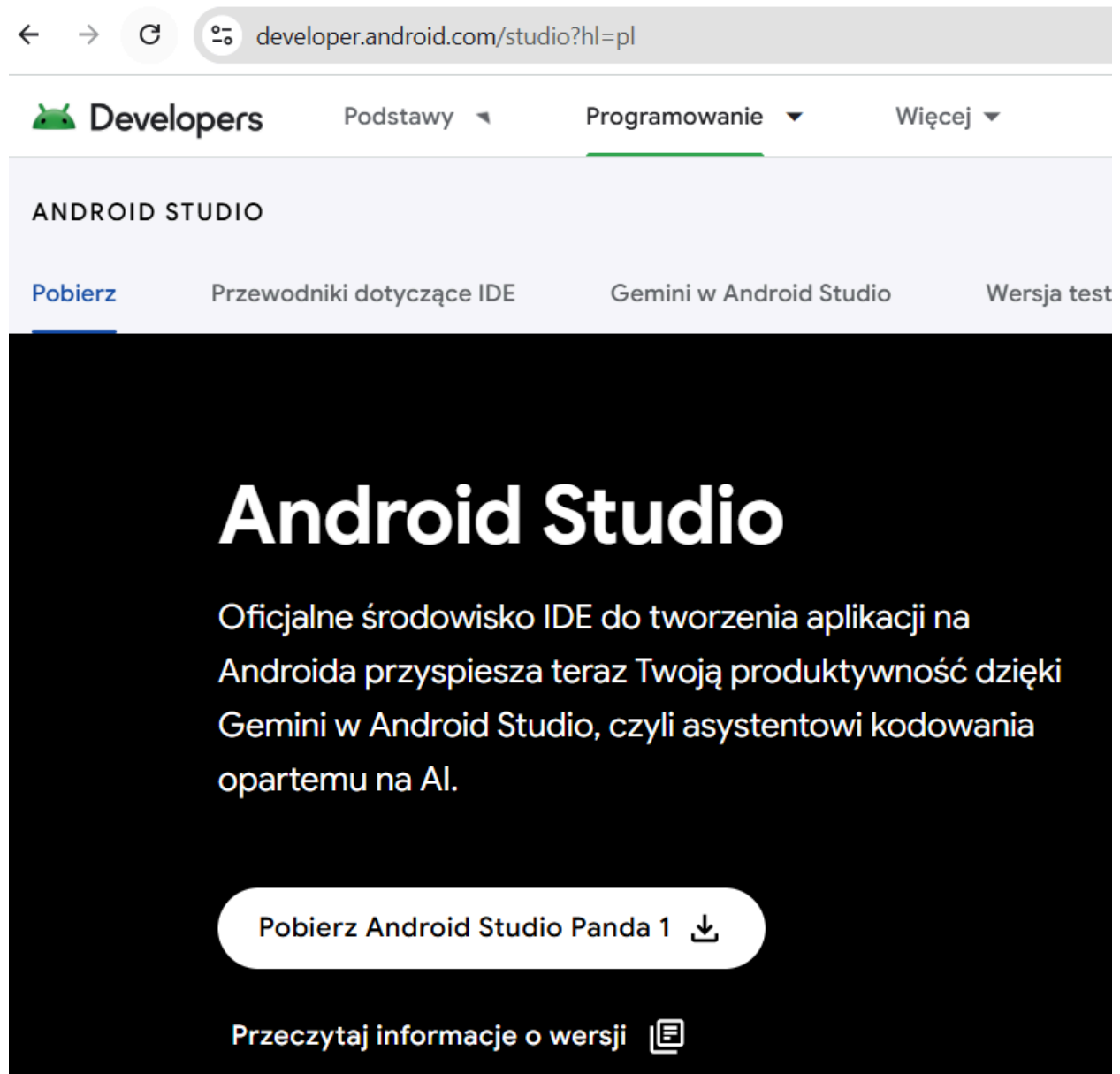


Projekt "MALIN"

Misja 1

- Konfiguracja IDE (ang. integrated development environment)
- Kurs programowania na Androida (cz. 1)
- Projekt

Konfiguracja IDE (ang. integrated development environment)



Kurs programowania na Androida (cz. 1)

Clean Architecture

Clean Architecture to podejście architektoniczne do projektowania systemów informatycznych, którego celem jest maksymalna separacja odpowiedzialności oraz uniezależnienie logiki biznesowej od szczegółów technologicznych (frameworków, baz danych, UI, infrastruktury).

Warstwa domenowa

- Może być współdzielona między platformami (web, desktop, Android, iOS)
- Use case'y
- Logika biznesowa
- Modele domenowe
- Interfejsy repozytoriów

Warstwa danych

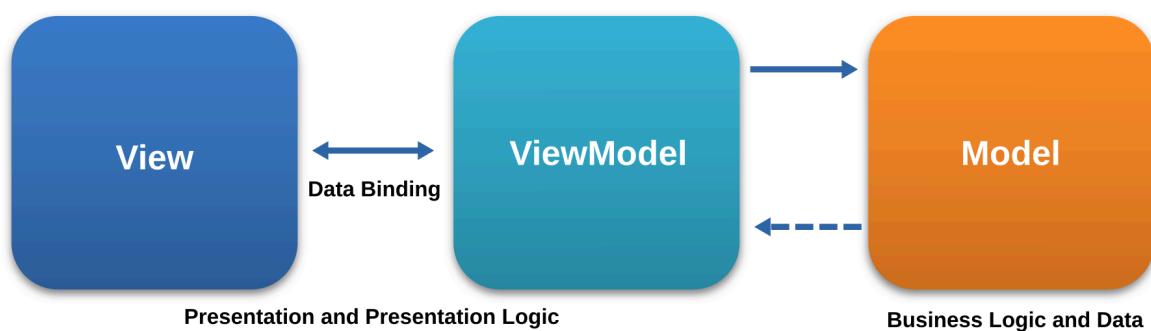
- Również może być współdzielona między platformami
- Implementacje repozytoriów
- Klient HTTP
- Bazy danych

Warstwa prezentacji

- Tylko kod związany bezpośrednio z UI

MVVM

Idea wzorca architektonicznego **MVVM (Model-View-ViewModel)** polega na oddzieleniu interfejsu użytkownika od logiki prezentacyjnej oraz umożliwienie efektywnego mechanizmu wiązania danych (data binding).



View - to warstwa wizualna, czyli UI;

ViewModel - pośrednik między View a Modelem; udostępnia dane w formie wygodnej dla UI;

Model - reprezentuje dane i logikę biznesową.

Podstawy Kotliny (cz. 1)

Zalecane narzędzie do nauki programowania w Kotlinie:

<https://play.kotlinlang.org/>

Deklaracja zmiennych

W Kotlinie zmienne deklarujemy za pomocą:

- **val** - wartość niezmienna (read-only)
- **var** - wartość zmienna

Przykład poprawnej deklaracji:

```
val hello = "hello"
```

Ogólna zasada: W Kotlinie preferujemy **val**, a **var** stosujemy tylko wtedy, gdy wartość faktycznie ma się zmieniać.

Błędne przykłady:

```
var hello: Int? = ""
```

```
String "hello" = hello
```

```
hello: String = "hello"
```

Poprawne sposoby zwiększania lub zmniejszania wartości zmiennych:

```
total++
```

```
total--
```

```
total = total + 1
```

Błędne sposoby zwiększania lub zmniejszania wartości zmiennych:

total - 1

Zagadka:

```
fun main() {  
    var x = 1  
    val y = x++ + ++x + x++  
    println("x = $x")  
    println("y = $y")  
}
```

Rozwiązanie zagadki:

`x++`

- post-increment (użyta wartość: 1; później `x` zwiększa się)
- wynik części: 1
- `x = 2`

`++x`

- pre-increment (najpierw zwiększa `x`; później używa wartości)
- `x = 3`
- wynik części: 3

`x++`

- znowu post-increment (użyta wartość: 3; później `x` zwiększa się)
- wynik części: 3
- `x = 4`

Reasumując:

`x = 4`

`y = 1 + 3 + 3`

`y = 7`

Komentarze w Kotlinie mogą być:

- jednolinijkowe: `// komentarz`
- wielolinijkowe: `/* komentarz */`

Komentarze są ignorowane przez kompilator.

Podstawowe typy danych w Kotlinie:

- String
- Int
- Boolean
- Double (64 bity; ok. 15-16 cyfr znaczących)
- Float (32 bity; ok. 6-7 cyfr znaczących)

Punktem startowym programu w Kotlinie jest funkcja `main()`.

Jeśli funkcja w Kotlinie nie określa jawnie zwracanego typu, domyślnym zwracanym typem jest `Unit`. W takim przypadku nie trzeba używać instrukcji `return`.

Wartość zwracaną przez funkcję można przypisać do zmiennej.

Typ zwracany musi być zgodny z deklaracją funkcji.

Funkcje mogą przyjmować `parametry`. `Argumenty` to wartości przekazywane do parametrów.

Dzieląc kod na funkcje, ułatwiamy jego utrzymanie.

Parametry funkcji mogą (ale nie muszą) mieć **wartości domyślnych**.
Dzięki **argumentom nazwanym** można zmieniać kolejność przekazywania argumentów.

Przykład:

```
fun greet(  
    name: String,  
    age: Int  
) { ... }
```

```
greet(age = 20, name = "Anna")
```

Instrukcja **if** pozwala sprawdzić warunek i wykonać kod tylko wtedy, gdy jest on prawdziwy.

Przykład:

```
fun main() {  
    val number = 100  
  
    if (number % 10 == 0) {  
        println("Podzielne przez 10")  
    } else {  
        println("Niepodzielne przez 10")  
    }  
}
```

```
Podzielne przez 10
```

Operatory:

```
fun main() {  
    val x = 5  
  
    println(x == 5)  
    println(x in 1..5)  
    println(x is Int)  
    println(x % 5)  
}
```



Co otrzymamy w ostatnim przypadku?

Cztery klasyczne filary **OOP (object-oriented programming)** to:

Abstrakcja

```
abstract class Animal {  
    abstract fun makeSound()  
}  
  
class Dog : Animal() {  
    override fun makeSound() = println("Woof!")  
}  
  
fun main() {  
    val a: Animal = Dog()  
    a.makeSound()  
}
```

Woof!

Dziedziczenie

```
open class Vehicle(val brand: String) {  
    fun start() = println("Starting...")  
}  
  
class Car(brand: String, val doors: Int) : Vehicle(brand)  
  
fun main() {  
    val car = Car("Toyota", 4)  
    car.start()      // odziedziczone  
    println(car.brand)  
}
```

```
Starting...  
Toyota
```

Polimorfizm

```
open class Shape {  
    open fun area(): Double = 0.0  
}  
  
class Circle(val r: Double) : Shape() {  
    override fun area() = Math.PI * r * r  
}  
  
fun printArea(shape: Shape) {  
    println(shape.area())  
}  
  
fun main() {  
    printArea(Circle(2.0))    // różna implementacja area()  
}
```

12.566370614359172

Można pisać kod, który działa na ogólnych typach (np. Shape) zamiast na konkretnych (np. Circle). Dzięki temu można łatwiej dodawać nowe klasy — np. Rectangle — bez ruszania istniejącego kodu.

Bez pomimorfizmu:

```
if (shape is Circle) { ... }  
else if (shape is Rectangle) { ... }  
... To koszmar w utrzymaniu.
```

Z polimorfizmem:

```
fun printArea(shape: Shape) {  
    println(shape.area())// żadnych ifów!  
}
```

Polimorfizm to możliwość wywoływania tych samych operacji na różnych obiektach, które realizują je na własny sposób.

W językach takich jak Kotlin czy C++ najczęściej osiąga się to poprzez dziedziczenie i nadpisywanie metod, choć istnieją też inne formy polimorfizmu, np. generics czy przeciążanie funkcji.

Enkapsulacja (ukrywanie danych)

```
class BankAccount {  
    private var balance = 25.9  
  
    fun deposit(amount: Double) {  
        if (amount > 0) balance += amount  
    }  
  
    fun getBalance(): Double = balance  
}  
  
fun main() {  
    val acc = BankAccount()  
    acc.deposit(100.0)  
    println(acc.getBalance()) |  
}
```

125.9

Modyfikatory dostępu w Kotlinie:

public - widoczne wszędzie

```
// Plik: Animal.kt
public class Animal {
    public fun makeSound() = println("Some sound")
}

// Plik: Main.kt
fun main() {
    val a = Animal()
    a.makeSound() // działa - public widoczne wszędzie
}

Some sound
```

private - widoczne tylko w obrębie pliku/klasy

```
class Person {
    private val secret = "Hasło123"

    fun reveal() = println(secret) // działa
}

fun main() {
    val p = Person()
    println(p.secret) // błąd - private
}

❗ Cannot access 'val secret: String': it is private in 'Person'.
```


protected - widoczne w klasie i jej podklasach

```
open class Vehicle {
    protected fun startEngine() = println("Engine started")
}

class Car : Vehicle() {
    fun drive() {
        startEngine() // działa - Car dziedziczy po Vehicle
    }
}

fun main() {
    val c = Car()
    c.startEngine() // błąd - protected
    c.drive()
}
```

❗ Cannot access 'fun startEngine(): Unit': it is protected in 'Vehicle'.

internal - widoczne w module

```
// Plik: ModuleA.kt (ten sam moduł)
internal class InternalService {
    internal fun run() = println("Running internal service")
}

// Plik: Main.kt (ten sam moduł)
fun main() {
    val s = InternalService()
    s.run() // działa - ten sam moduł
}
```

Running internal service

Słowo kluczowe **super** służy do wywoływania metod z klasy nadrzędnej:

```
open class A {  
    open fun hello() = println("Hello from A")  
}  
  
class B : A() {  
    override fun hello() {  
        super.hello()    // wywołanie metody z A  
        println("Hello from B")  
    }  
}  
  
fun main() {  
    val b = B()  
    b.hello()  
}
```

```
Hello from A  
Hello from B
```

Interfejs – kontrakt dla klasy. Definiuje metody lub właściwości, które klasa musi zaimplementować.

```
interface Clickable {  
    fun click()  
    fun showHint() = println("Kliknij, aby aktywować")  
}  
  
class Button : Clickable {  
    override fun click() {  
        println("Button: Wykonuję akcję...")  
    }  
}  
  
class Checkbox : Clickable {  
    var checked = false  
  
    override fun click() {  
        checked = !checked  
        println("Checkbox: checked = $checked")  
    }  
}  
  
fun main() {  
    val checkbox = Checkbox()  
    checkbox.showHint()  
    checkbox.click()  
  
    val button = Button()  
    button.showHint()  
    button.click()  
}
```

```
Kliknij, aby aktywować  
Checkbox: checked = true  
Kliknij, aby aktywować  
Button: Wykonuję akcję...
```

Typ **nullable** (?) oznacza, że wartość może być null.

Przykład:

```
var lastName: String
```

```
var profilePhotoUrl: String?
```

Operator bezpiecznego wywołania ?. umożliwia wywoływanie metody tylko wtedy, gdy obiekt nie jest nulle.

```
fun main() {  
    val name: String? = null  
    println(name.length)  
}
```

❗ Only safe (?.) or non-null asserted (!!) calls are allowed on a nullable receiver of type 'String'

```
fun main() {  
    val name: String? = null  
    println(name?.length)  
}
```

```
null
```

W Kotlinie funkcje mogą być przekazywane jako argumenty innych funkcji:

```
fun processNumbers(
    numbers: List<Int>,
    operation: (Int) -> Int
): List<Int> {
    return numbers.map { operation(it) }
}

fun main() {
    val nums = listOf(1, 2, 3, 4)

    val doubled = processNumbers(nums) { it * 2 }
    val squared = processNumbers(nums) { it * it }
    val incremented = processNumbers(nums) { it + 1 }

    println(doubled)
    println(squared)
    println(incremented)
}
```

```
[2, 4, 6, 8]
[1, 4, 9, 16]
[2, 3, 4, 5]
```

W Kotlinie funkcje mogą być zwracane z funkcji:

```

fun multiplier(factor: Int): (Int) -> Int {
    return { number -> number * factor }
}

fun main() {
    val double = multiplier(2)
    val triple = multiplier(3)
    val tenTimes = multiplier(10)

    println(double(5))
    println(triple(5))
    println(tenTimes(5))
}

```

```

10
15
50

```

W Kotlinie **lambda** jest **literałem** funkcji, czyli wartością funkcji zapisaną w kodzie, tak jak:

- "tekst" jest literałem stringa,
- 42 jest literałem liczby,
- true jest literałem boolean,
- { number -> number * factor } jest literałem funkcji.

Projekt

W każdym zespole jedna osoba tworzy projekt i wysyła go do repozytorium.

Misja 2

- Kurs programowania na Androida (cz. 2)
- Implementacja mechanizmu pozycjonowania (cz. 1)

Jetpack Compose (cz. 1)

```
@Composable
fun BeaconList(
    viewModel: MainViewModel,
    modifier: Modifier
) {
    val beacons = viewModel.beacons.collectAsState()

    LazyColumn(modifier) {
        items(beacons.value) { beacon: Beacon ->
            Text(
                text = beacon.name,
                textAlign = TextAlign.Center,
                modifier = Modifier.fillMaxWidth()
            )
        }
    }
}
```

Jetpack Compose to nowoczesne narzędzie do tworzenia interfejsów użytkownika na Androidzie w sposób deklaratywny.

Composable functions to podstawowe elementy budujące UI. Każdy element interfejsu (np. tekst, przycisk) jest funkcją.

Każda funkcja UI musi być oznaczona adnotacją **@Composable** aby Compose wiedział, że ma ją renderować.

Podstawowe layouty w Compose:

- Column (układ pionowy)
- Row (układ poziomy)
- Box (nakładanie elementów)

Resource Manager to narzędzie w Android Studio do zarządzania grafikami, stringami, kolorami itd.

```
Text(text = stringResource(R.string.app_name))
```

```
Box(  
    modifier = Modifier  
        .background(colorResource(R.color.primaryColor))  
)
```

```
Image(  
    painter = painterResource(R.drawable.ic_launcher_foreground),  
    contentDescription = null
```


)

```
val padding = dimensionResource(R.dimen.default_padding)
```

```
Box(modifier = Modifier.padding(padding))
```

```
val items = stringArrayResource(R.array.menu_items)
```

Klasa **R** zawiera identyfikatory wszystkich zasobów projektu (np. **R.drawable.app_logo**).

Implementacja mechanizmu pozycjonowania (cz. 1)

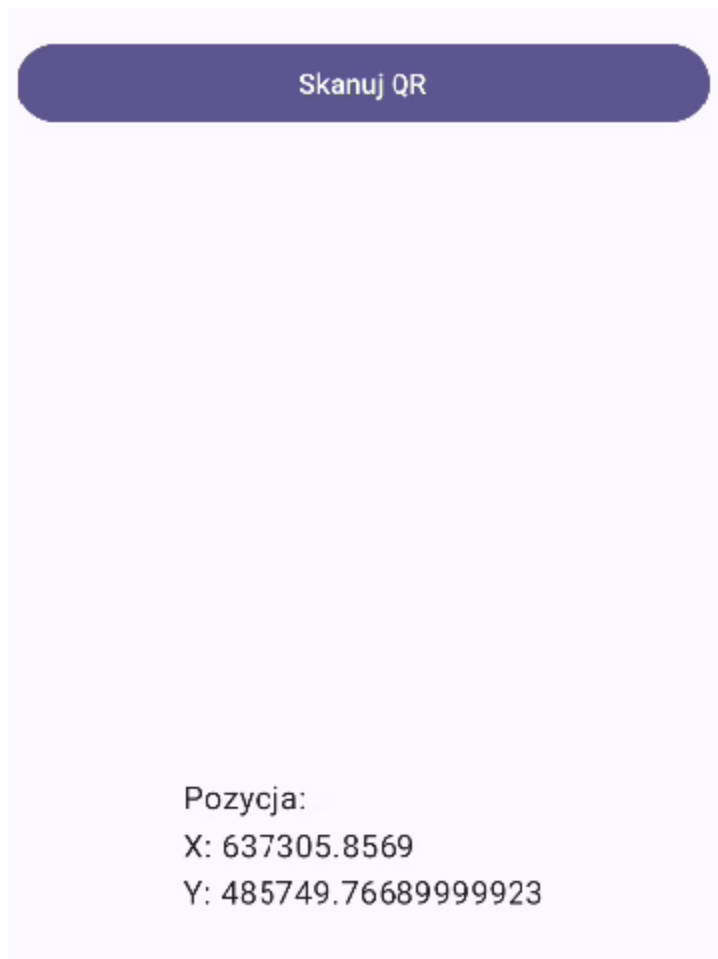
Skanowanie kodów QR

- <https://developers.google.com/ml-kit/vision/barcode-scanning/code-scanner>
- <https://github.com/journeyapps/zxing-android-embedded>

Misja 3

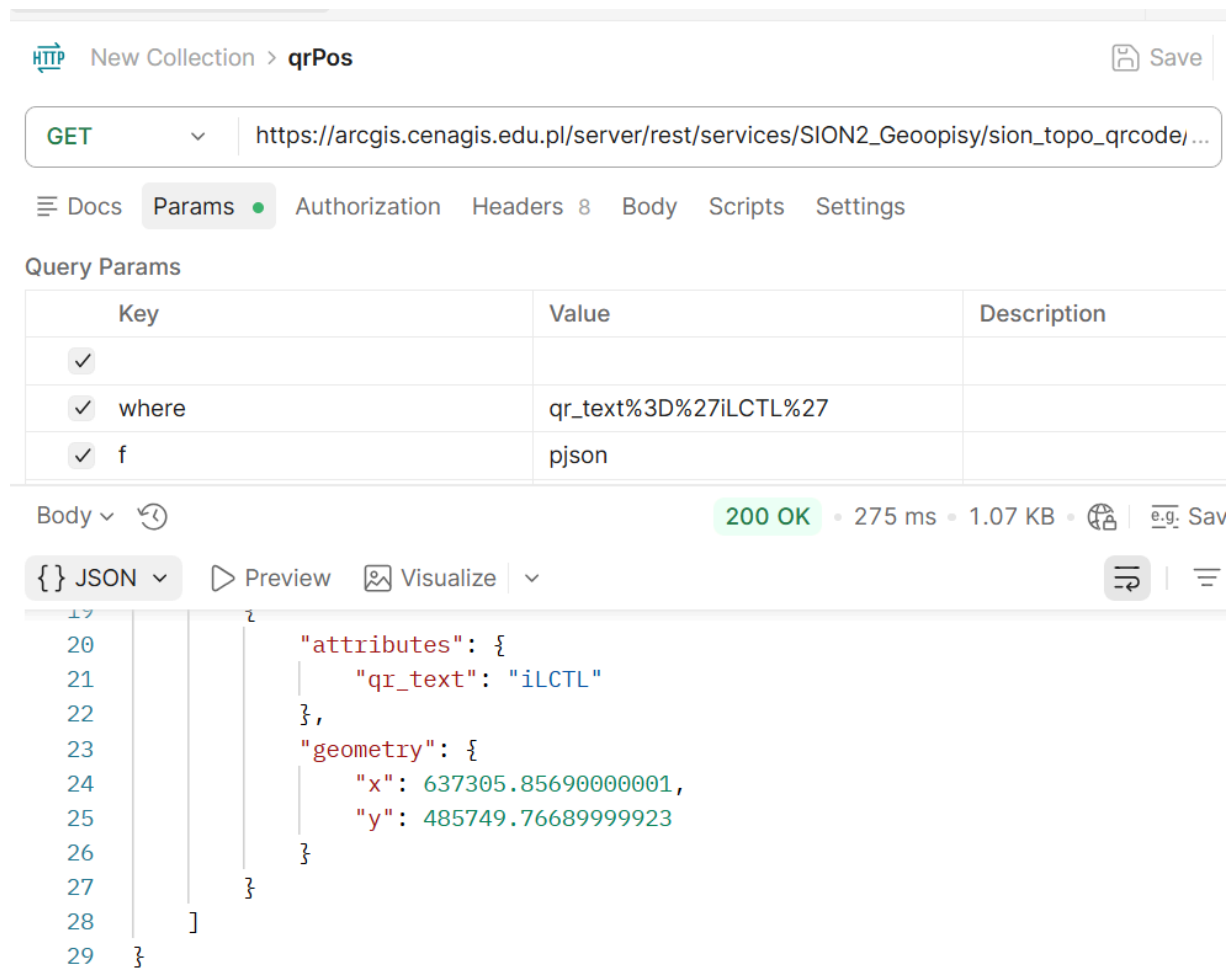
- Implementacja mechanizmu pozycjonowania (cz. 2)

Nasz dzisiejszy cel:



Testujemy API w POSTMAN-ie

- <https://www.postman.com/>



New Collection > qrPos Save

GET https://arcgis.cenagis.edu.pl/server/rest/services/SION2_Geoopisy/sion_topo_qrcode/...

Docs Params Authorization Headers 8 Body Scripts Settings

Query Params

Key	Value	Description
☒		
☒ where	qr_text%3D%27iLCTL%27	
☒ f	pjson	

Body 200 OK • 275 ms • 1.07 KB • e.g. Sav

{ JSON Preview Visualize

```
19 {
20   "attributes": {
21     "qr_text": "iLCTL"
22   },
23   "geometry": {
24     "x": 637305.856900000001,
25     "y": 485749.76689999923
26   }
27 }
28 ]
29 }
```

Usługa typu GET zwracająca pozycję w układzie EPSG:2180 (PUWG 1992) na podstawie kodu QR:

https://arcgis.cenagis.edu.pl/server/rest/services/SION2_Geoopisy/sion_topo_qrcode/MapServer/0/query?&where=qr_text%3D%27iLCTL%27&f=pjson

Retrofit

- <https://square.github.io/retrofit/>

Podstawy programowania asynchronicznego

- <https://kotlinlang.org/docs/coroutines-basics.html>

sealed class

```
package pl.lipov.malin.common

sealed class ResultState<out T> {
    data class Success<T>(val data: T) :
        ResultState<T>()
    data class Error(val throwable: Throwable) :
        ResultState<Nothing>()
    object Loading : ResultState<Nothing>()
}
```

Stosuj sealed class + typy generyczne zawsze, gdy Twoje dane mogą być w różnych stanach.

Zarządzanie stanem

- <https://developer.android.com/kotlin/flow/stateflow-and-shared-flow>
- [https://developer.android.com/reference/kotlin/androidx/compose/runtime/collectAsState.composable#\(kotlinx.coroutines.flow.StateFlow\).collectAsState\(kotlin.coroutines.CoroutineContext\)](https://developer.android.com/reference/kotlin/androidx/compose/runtime/collectAsState.composable#(kotlinx.coroutines.flow.StateFlow).collectAsState(kotlin.coroutines.CoroutineContext))